

File Descriptor (M. Harshika - 2520090047)

- A **file descriptor (FD)** is simply an integer number that the operating system gives to a process when it opens something.
- The operating system assigns the lowest available file descriptor number when you open a resource.

Think of it as a ticket number.

Suppose you tell Linux:

Open **student.txt**

Linux may say:

Okay, I will give you ticket number 3.

So:

Student.txt → **FD3**

Your program doesn't have to keep saying "**student.txt**" every time.

It can simply say FD 3 to refer to that open file.

Similarly, Linux gives your program a number:

File → File Descriptor

For example: *student.txt* → 3

That number 3 is the file descriptor.

Every normal process starts with three standard file descriptors:

0 → *Standard Input*

1 → *Standard Output*

2 → Standard Error

0 → stdin → Where do I GET input?

1 → stdout → Where do I SEND normal output?

2 → stderr → Where do I SEND error output?

We normally write them as:

STDIN_FILENO

STDOUT_FILENO

STDERR_FILENO

Number	Name	Meaning
0	STDIN_FILENO	Input
1	STDOUT_FILENO	Output
2	STDERR_FILENO	Error output

Program 1: stdout — Standard Output

```
#include <unistd.h>
int main()
{
    write(STDOUT_FILENO, "Hello World\n", 12);
    return 0;
}
```

The screenshot shows a terminal window on the left and a code editor on the right. The terminal shows the user creating a directory, changing to it, editing a file, compiling it with gcc, and running it, which outputs "Hello World". The code editor shows the source code for program1.c, which includes <unistd.h>, defines main(), and uses write() to output "Hello World\n" to STDOUT_FILENO.

```
harshika@siri: ~/FileDescriptc
harshika@siri:~$ mkdir FileDescriptors
harshika@siri:~$ cd FileDescriptors
harshika@siri:~/FileDescriptors$ gedit program1.c

** (gedit:633): WARNING **: 06:43:20.529: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:633): WARNING **: 06:43:20.529: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program1.c -o program1
harshika@siri:~/FileDescriptors$ ./program1
Hello World
```

```
1 #include <unistd.h>
2 int main()
3 {
4     write(STDOUT_FILENO, "Hello World\n", 12);
5     return 0;
6 }
```

Program 2: stdin — Standard Input

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    char buffer[20];
```

```
    read(STDIN_FILENO, buffer, sizeof(buffer));
```

```
    write(STDOUT_FILENO, buffer, sizeof(buffer));
```

```
    return 0;
```

```
}
```

The screenshot shows a terminal window on the left and a code editor on the right. The terminal shows the user creating a file, editing it, compiling it with gcc, and running it, which outputs "Hello" twice. The code editor shows the source code for program2.c, which includes <unistd.h>, defines main(), and uses read() and write() to read from and write to STDIN_FILENO and STDOUT_FILENO respectively.

```
: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gedit program2.c

** (gedit:1517): WARNING **: 06:44:53.459: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:1517): WARNING **: 06:44:53.459: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program2.c -o program2
harshika@siri:~/FileDescriptors$ ./program2
Hello
Hello
```

```
1 #include <unistd.h>
2 int main()
3 {
4     char buffer[20];
5     read(STDIN_FILENO, buffer, sizeof(buffer));
6     write(STDOUT_FILENO, buffer, sizeof(buffer));
7     return 0;
8 }
```

Program 3: Proper stdin + stdout

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    char buffer[100];
```

```
    int n;
```

```
    printf("Enter something: ");
```

```
    n = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);
```

```
    if (n > 0)
```

```

{
    buffer[n] = '\0';
    write(STDOUT_FILENO, buffer, n);
}
return 0;
}

```

The screenshot shows a terminal window on the left and a code editor on the right. The terminal window displays the following commands and output:

```

harshika@siri: ~/FileDescriptors
: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$
harshika@siri:~/FileDescriptors$ gedit program3.c

** (gedit:2210): WARNING **: 06:46:23.095: Could not load Peas repository: T
ypelib file for namespace 'Peas', version '1.0' not found

** (gedit:2210): WARNING **: 06:46:23.096: Could not load PeasGtk repository
: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program3.c -o program3
harshika@siri:~/FileDescriptors$ ./program3
Hii!!
Hii!!
harshika@siri:~/FileDescriptors$ Fedit program3.c

** (gedit:2916): WARNING **: 06:47:30.975: Could not load Peas repository: T
ypelib file for namespace 'Peas', version '1.0' not found

** (gedit:2916): WARNING **: 06:47:30.975: Could not load PeasGtk repository
: Typelib file for namespace 'PeasGtk', version '1.0' not found

```

The code editor on the right shows the contents of program3.c:

```

1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     char buffer[100];
6     int n;
7     printf("Enter something: ");
8     n = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);
9     if (n > 0)
10    {
11        buffer[n] = '\0';
12        write(STDOUT_FILENO, buffer, n);
13    }
14    return 0;
15 }

```

Program 4: stderr — Standard Error

```

#include <unistd.h>
int main()
{
    write(STDERR_FILENO, "This is an error\n", 17);
    return 0;
}

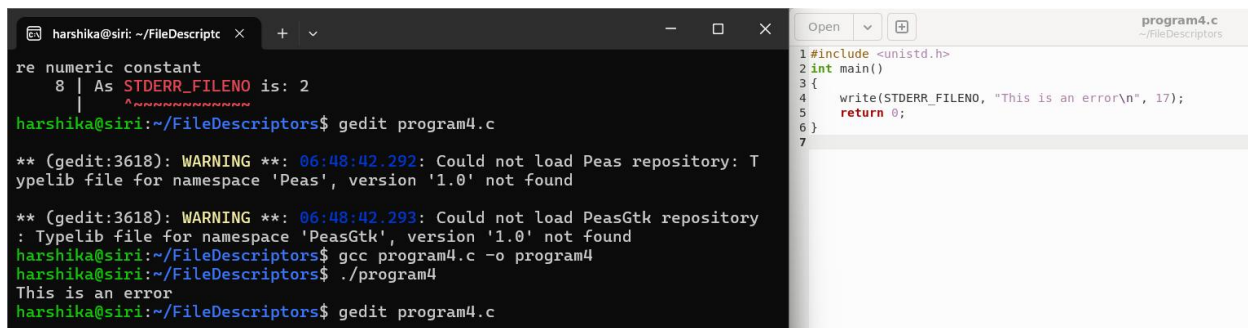
```

As STDERR_FILENO is: 2

write(STDERR_FILENO, "This is an error\n", 17) is equivalent to:

```
write(2, "This is an error\n", 17);
```

You will see output as: This is an error.



```
harshika@siri: ~/FileDescriptc x + v
re numeric constant
8 | As STDERR_FILENO is: 2
harshika@siri:~/FileDescriptors$ gedit program4.c

** (gedit:3618): WARNING **: 06:48:42.292: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:3618): WARNING **: 06:48:42.293: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program4.c -o program4
harshika@siri:~/FileDescriptors$ ./program4
This is an error
harshika@siri:~/FileDescriptors$ gedit program4.c
```

```
program4.c
~/FileDescriptors
1 #include <unistd.h>
2 int main()
3 {
4     write(STDERR_FILENO, "This is an error\n", 17);
5     return 0;
6 }
7
```

Program 5: stderr for an Actual Error

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    int fd;
```

```
    fd = open("doesnotexist.txt", O_RDONLY);
```

```
    if (fd == -1)
```

```
    {
```

```
        perror("open");
```

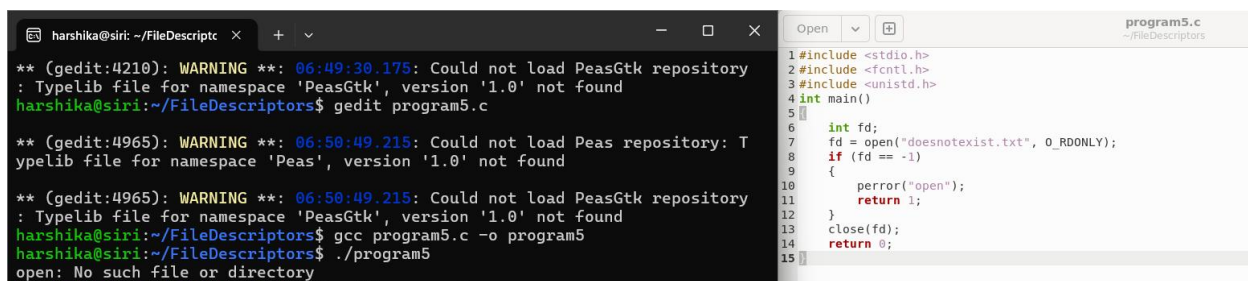
```
        return 1;
```

```
    }
```

```
    close(fd);
```

```
    return 0;
```

```
}
```



```
harshika@siri: ~/FileDescriptc x + v
** (gedit:4210): WARNING **: 06:49:30.175: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gedit program5.c

** (gedit:4965): WARNING **: 06:50:49.215: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:4965): WARNING **: 06:50:49.215: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program5.c -o program5
harshika@siri:~/FileDescriptors$ ./program5
open: No such file or directory
```

```
program5.c
~/FileDescriptors
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4 int main()
5 {
6     int fd;
7     fd = open("doesnotexist.txt", O_RDONLY);
8     if (fd == -1)
9     {
10         perror("open");
11         return 1;
12     }
13     close(fd);
14     return 0;
15 }
```

Program 6 Creating file using file descriptor

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    fd = open("student.txt", O_CREAT | O_WRONLY, 0644);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }
    printf("File created successfully!\n");
    printf("File descriptor = %d\n", fd);
    close(fd);
    return 0;
}
```

gcc create.c -o create

./create

Output:

File created successfully! File descriptor = 3

fd = open("student.txt", O_CREAT | O_WRONLY, 0644); describes the information below:

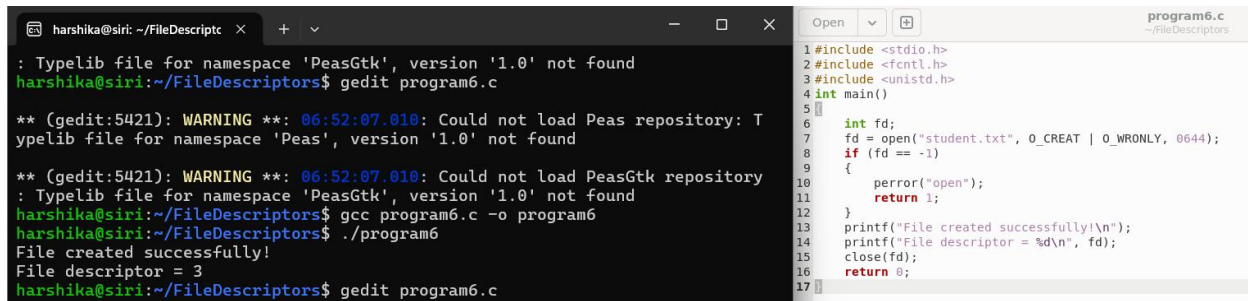
"student.txt" → file name

O_CREAT → create if it doesn't exist

O_WRONLY → open for writing

0644 → file permissions

fd → stores the file descriptor



```
harshika@siri: ~/FileDescriptc x + v - □ x
: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gedit program6.c

** (gedit:5421): WARNING **: 06:52:07.010: Could not load Peas repository: T
ypelib file for namespace 'Peas', version '1.0' not found

** (gedit:5421): WARNING **: 06:52:07.010: Could not load PeasGtk repository
: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program6.c -o program6
harshika@siri:~/FileDescriptors$ ./program6
File created successfully!
File descriptor = 3
harshika@siri:~/FileDescriptors$ gedit program6.c

1#include <stdio.h>
2#include <fcntl.h>
3#include <unistd.h>
4int main()
5{
6    int fd;
7    fd = open("student.txt", O_CREAT | O_WRONLY, 0644);
8    if (fd == -1)
9    {
10        perror("open");
11        return 1;
12    }
13    printf("File created successfully!\n");
14    printf("File descriptor = %d\n", fd);
15    close(fd);
16    return 0;
17}
```

Program 7

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    fd = open("student.txt", O_WRONLY | O_TRUNC);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }
    write(fd, "New Data\n", 9);
    close(fd);
    return 0;
}
```

If a file already has data, O_TRUNC removes all its existing contents when you open it for writing.

First, suppose student.txt contains:

Hello

Welcome

Operating Systems

After open() with O_TRUNC:
Student.txt contains (empty)

DUP () : duplicate a file descriptor.

dup() creates a new file descriptor that refers to the same underlying open file or I/O resource as the original descriptor.

Suppose:

```
int fd1 = open("student.txt", O_WRONLY);
```

If the OS gives:

```
fd1 = 3
```

then:

FD 3 → student.txt

Now we do:

```
int fd2 = dup(fd1);
```

The OS creates another file descriptor, usually:

```
fd2 = 4
```

Now both FD 3 and FD 4 refer to the same open file.

Program 7 - Understand dup()

```
#include <stdio.h>  
#include <fcntl.h>
```

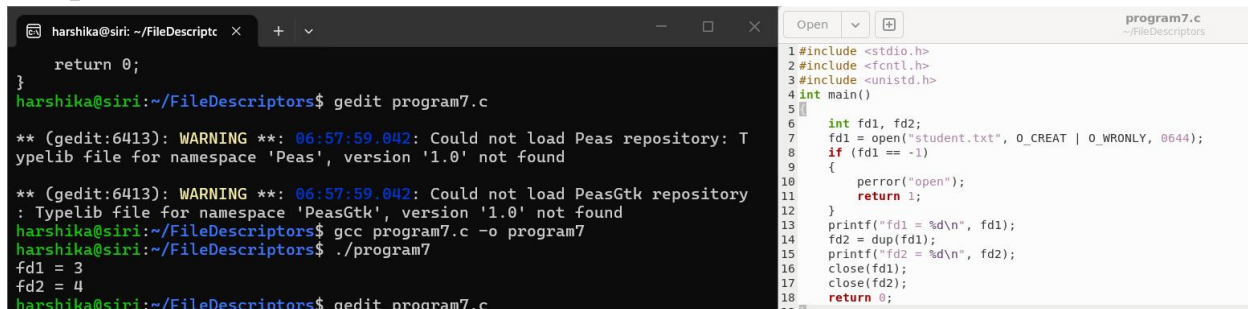


```

#include <unistd.h>
int main()
{
    int fd1, fd2;
    fd1 = open("student.txt", O_CREAT | O_WRONLY, 0644);
    if (fd1 == -1)
    {
        perror("open");
        return 1;
    }
    printf("fd1 = %d\n", fd1);
    fd2 = dup(fd1);
    printf("fd2 = %d\n", fd2);
    close(fd1);
    close(fd2);
    return 0;
}

```

Output: fd1 = 3 fd2 = 4



```

harshika@siri: ~/FileDescriptc  x  +  v  Open  program7.c
~/FileDescriptc
return 0;
}
harshika@siri:~/FileDescriptors$ gedit program7.c
** (gedit:6413): WARNING **: 06:57:59.042: Could not load Peas repository: T
ypelib file for namespace 'Peas', version '1.0' not found
** (gedit:6413): WARNING **: 06:57:59.042: Could not load PeasGtk repository
: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~/FileDescriptors$ gcc program7.c -o program7
harshika@siri:~/FileDescriptors$ ./program7
fd1 = 3
fd2 = 4
harshika@siri:~/FileDescriptors$ gedit program7.c
1#include <stdio.h>
2#include <fcntl.h>
3#include <unistd.h>
4int main()
5{
6    int fd1, fd2;
7    fd1 = open("student.txt", O_CREAT | O_WRONLY, 0644);
8    if (fd1 == -1)
9    {
10        perror("open");
11        return 1;
12    }
13    printf("fd1 = %d\n", fd1);
14    fd2 = dup(fd1);
15    printf("fd2 = %d\n", fd2);
16    close(fd1);
17    close(fd2);
18    return 0;
19}

```